



Do An CS 4 - sadas

VKU_Đồ án cơ sở 4 (Trường Đại học Công nghệ Thông tin và Truyền thông Việt - Hàn)



messages.pdf_cover_qr_code_label

**TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN &
TRUYỀN THÔNG VIỆT-HÀN
KHOA KHOA HỌC MÁY TÍNH**



ĐỒ ÁN CƠ SỞ 4

ĐỀ TÀI: XÂY DỰNG ỨNG DỤNG TRÒ CHƠI CỜ VUA

Sinh viên thực hiện : **DƯƠNG THỊ LIÊN**

Giảng viên hướng dẫn : **THS.TRỊNH THỊ NGỌC LINH**

Đà Nẵng, tháng 12 năm 2023

**TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN &
TRUYỀN THÔNG VIỆT-HÀN
KHOA KHOA HỌC MÁY TÍNH**



ĐỒ ÁN CƠ SỞ 4

ĐỀ TÀI: XÂY DỰNG ỨNG DỤNG TRÒ CHƠI CỜ VUA

Sinh viên : **DƯƠNG THỊ LIÊN**

Mã sinh viên : **21IT621**

Giảng viên hướng dẫn : **THS.TRỊNH THỊ NGỌC LINH**

Đà Nẵng, tháng 12 năm 2023

LỜI MỞ ĐẦU

Hiện nay, công nghệ ngày càng phát triển, dẫn đến công nghệ thông tin phát triển theo. Trong số đó các ứng dụng game đã thu hút sự quan tâm và yêu thích của mọi người với sự tiện lợi và giải trí mà nó mang lại. Do đó việc xây dựng các ứng dụng giải trí đang là một ngành công nghệ mới đầy tiềm năng và hứa hẹn phát triển vượt bậc.

Cờ vua, một trò chơi đồng thời mang tính giải trí và chiến thuật cao, đã tồn tại hàng thế kỷ và vẫn giữ vững sự phổ biến đến ngày nay. Việc phát triển một ứng dụng game cờ vua không chỉ mang lại niềm vui và thách thức cho người chơi mà còn mở ra cơ hội khám phá và ứng dụng các công nghệ mới như Trí tuệ Nhân tạo.

Trí tuệ Nhân tạo đã đạt được những tiến bộ đáng kể trong thập kỷ gần đây. Khả năng của AI để học hỏi, phân tích và đưa ra quyết định thông minh đã tạo ra những cơ hội mới và thú vị cho lĩnh vực game. Trong ứng dụng game cờ vua với công nghệ AI, chúng tôi tập trung vào việc phát triển một đối thủ máy tính thông minh, có khả năng chơi cờ với người chơi và cung cấp gợi ý chiến thuật thông minh.

Trong báo cáo này, chúng tôi sẽ trình bày về quá trình phát triển ứng dụng game cờ vua với ứng dụng AI, từ việc thiết kế giao diện người dùng hấp dẫn và thân thiện đến việc xây dựng một hệ thống AI thông minh để chơi cờ với người dùng. Chúng tôi cũng sẽ đề cập đến các phương pháp và thuật toán trong việc xây dựng đối thủ máy tính thông minh, từ việc phân tích vị trí trên bàn cờ đến việc đưa ra các quyết định tối ưu.

Hy vọng rằng dự án này sẽ mang lại sự thú vị và hứng thú cho người chơi cờ vua và cung cấp thông tin hữu ích về việc kết hợp giữa game cờ vua và Trí tuệ Nhân tạo. Chúng tôi tin rằng ứng dụng này sẽ mang đến một trải nghiệm chơi cờ vua mới mẻ và tạo ra một thách thức thú vị cho người chơi.

NHẬN XÉT

(Của giảng viên hướng dẫn)

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

Chữ kí của giảng viên

ThS.Trịnh Thị Ngọc Linh

LỜI CẢM ƠN

Trên thực tế không có sự thành công nào mà không gắn liền với những sự hỗ trợ, giúp đỡ dù ít hay nhiều, dù trực tiếp hay gián tiếp của người khác. Trong suốt thời gian từ khi bắt đầu học tập ở giảng đường đại học đến nay, em đã nhận được rất nhiều sự quan tâm, giúp đỡ của quý Thầy Cô, gia đình và bạn bè.

Với lòng biết ơn sâu sắc nhất, em xin gửi đến quý Thầy Cô ở Trường Đại học Công nghệ Thông tin và Truyền thông Việt – Hàn đã cùng với tri thức và tâm huyết của mình để truyền đạt vốn kiến thức quý báu cho chúng em trong suốt thời gian học tập tại trường. Và đặc biệt, trong học kỳ này, Khoa đã tổ chức cho chúng em được tiếp cận với môn học mà theo em là rất hữu ích đối với các sinh viên thuộc các chuyên ngành. Đó là môn học “Đồ án cơ sở 4”

Em xin chân thành cảm ơn ThS.Trịnh Thị Ngọc Linh đã tận tâm hướng dẫn chúng em qua từng buổi học trên lớp cũng như những buổi nói chuyện, thảo luận về lĩnh vực sáng tạo trong nghiên cứu khoa học. Nếu không có những lời hướng dẫn, dạy bảo của cô thì em nghĩ bài thu hoạch này của em rất khó có thể hoàn thiện được. Một lần nữa, em xin chân thành cảm ơn cô.

MỤC LỤC

LỜI MỞ ĐẦU.....	1
NHẬN XÉT.....	2
LỜI CẢM ƠN.....	3
CHƯƠNG 1. GIỚI THIỆU VỀ NGÔN NGỮ PYTHON.....	5
1.1 Giới thiệu về ngôn ngữ python.....	6
1.2 Lịch sử phát triển.....	7
1.3 Các tính năng của Python.....	8
1.4 Ứng dụng Pycharm.....	9
1.5 Các tính năng của Pycharm.....	9
CHƯƠNG 2: TỔNG QUAN VỀ TRÒ CHƠI CỜ VUA.....	11
2.1 Giới thiệu trò chơi.....	11
2.2 Các quy tắc của trò chơi.....	11
2.3 GDD (Game Design Document).....	16
CHƯƠNG 3. PHÂN TÍCH VÀ THIẾT KẾ GAME.....	18
3.1 Sơ đồ lớp.....	18
3.2 Cơ sở lý thuyết xây dựng Chess bot.....	18
3.2.1 Ý tưởng đưa ra nước đi tối ưu.....	18
3.2.2 Các thuật toán để đưa ra nước đi tối ưu.....	18
3.3 Thiết kế logic cho Chess bot.....	20
3.3.1 Hệ số lực của quân cờ.....	20
3.3.2 Cây trò chơi.....	22
3.3.3 Thuật toán Negamax.....	23
3.3.4 Cắt tỉa Alpha-Beta.....	24
CHƯƠNG 4. THIẾT KẾ GIAO DIỆN.....	26
4.1 Giao diện trò chơi.....	26
4.2 Giao diện thông báo không có quân cờ được chọn.....	26
4.3 Giao diện thông báo nước đi không hợp lệ.....	27
4.4 Giao diện chiến thắng.....	28
4.5 Giao diện gợi ý nước đi.....	28
CHƯƠNG 5: KẾT LUẬN.....	30
5.1 Ưu điểm.....	30
5.2 Nhược điểm.....	30
5.3 Hướng phát triển.....	30
TÀI LIỆU THAM KHẢO.....	31

DANH MỤC HÌNH ẢNH

HÌNH 1: NGÔN NGỮ PYTHON.....	6
HÌNH 2: MINH HỌA NƯỚC ĐI QUÂN XE.....	12
HÌNH 3: MINH HỌA NƯỚC ĐI QUÂN TƯỢNG.....	12
HÌNH 4: MINH HỌA NƯỚC ĐI QUÂN HẬU.....	13
HÌNH 5: MINH HỌA NƯỚC ĐI QUÂN MÃ.....	13
HÌNH 6: MINH HỌA NƯỚC ĐI QUÂN TỐT.....	14
HÌNH 7: MINH HỌA QUÂN TỐT THẮNG CẤP.....	15
HÌNH 8: MINH HỌA NƯỚC ĐI QUÂN VUA.....	16
HÌNH 9: GAME FLOW.....	16
HÌNH 10: BÀN CỜ TIÊU CHUẨN.....	17
HÌNH 11: CÁC QUÂN CỜ VUA.....	17
HÌNH 12: HỆ SỐ LỰC CỦA MỖI QUÂN CỜ.....	20
HÌNH 13: BẢNG GIÁ TRỊ VỊ TRÍ.....	21
HÌNH 14: GIÁ TRỊ VỊ TRÍ CỦA QUÂN CỜ.....	22
HÌNH 15: MINH HỌA CÂY TRÒ CHƠI.....	22
HÌNH 16: MINH HỌA THUẬT TOÁN NEGAMAX.....	24
HÌNH 17: MINH HỌA VỀ THUẬT TOÁN CẮT TỈA ALPHA-BETA.....	24
HÌNH 18: GIAO DIỆN TRÒ CHƠI.....	26
HÌNH 19: GIAO DIỆN THÔNG BÁO KHÔNG CÓ QUÂN CỜ ĐƯỢC CHỌN.....	27
HÌNH 20: GIAO DIỆN CHIẾN THẮNG.....	28
HÌNH 21: GIAO DIỆN GỢI Ý NƯỚC ĐI.....	29

CHƯƠNG 1. GIỚI THIỆU VỀ NGÔN NGỮ PYTHON

1.1 Giới thiệu về ngôn ngữ python



Hình 1: Ngôn ngữ python

Python là một ngôn ngữ lập trình cao cấp, thông dịch và đa mục đích. Nó được phát triển bởi Guido van Rossum và lần đầu tiên được ra mắt vào năm 1991. Python nổi tiếng với cú pháp đơn giản, dễ đọc và dễ hiểu, giúp người lập trình tập trung vào việc giải quyết vấn đề thay vì lo lắng về các chi tiết cú pháp phức tạp.

Một trong những đặc điểm nổi bật của Python là sự linh hoạt. Nó hỗ trợ nhiều phong cách lập trình, từ lập trình hướng đối tượng cho đến lập trình hàm và lập trình cấu trúc. Điều này cho phép người lập trình sử dụng Python trong nhiều lĩnh vực khác nhau, bao gồm phân tích dữ liệu, trí tuệ nhân tạo, phát triển web, tự động hóa và nhiều ứng dụng khác.

Python có một cộng đồng lớn và sôi động, với hàng ngàn thư viện và framework phong phú. Các thư viện như NumPy, Pandas và Matplotlib cung cấp các công cụ mạnh mẽ cho tính toán khoa học và xử lý dữ liệu. Flask và Django là hai framework phổ biến trong việc phát triển ứng dụng web. TensorFlow và PyTorch là hai thư viện phổ biến trong lĩnh vực trí tuệ nhân tạo.

Python cũng được coi là ngôn ngữ dễ học và dễ bắt đầu. Với cú pháp đơn giản và tài liệu phong phú, người mới bắt đầu có thể nhanh chóng tiếp cận và xây dựng các ứng dụng đơn giản. Đồng thời, Python cũng là ngôn ngữ mạnh mẽ và linh hoạt đối với các lập trình viên có kinh nghiệm.

Tổng quan về Python cho thấy nó là một ngôn ngữ lập trình phổ biến và mạnh mẽ, được sử dụng rộng rãi trong nhiều lĩnh vực khác nhau. Với cú pháp đơn giản, người lập trình có thể tập trung vào giải quyết vấn đề và phát triển các ứng dụng hiệu quả.

1.2 Lịch sử phát triển

Python bắt đầu vào những năm đầu của thập kỷ 1990 khi Guido van Rossum, một nhà lập trình người Hà Lan, bắt đầu phát triển ngôn ngữ này. Ban đầu, ý tưởng của Guido là tạo ra một ngôn ngữ lập trình dễ đọc và hiểu, có thể tương tác tốt với hệ thống Unix. Ông đã đặt tên cho ngôn ngữ này theo tên bộ phim hài truyền hình "Monty Python's Flying Circus".

Phiên bản đầu tiên của Python, Python 0.9.0, được phát hành vào tháng 2 năm 1991. Từ đó, Python nhanh chóng thu hút sự quan tâm và ủng hộ từ cộng đồng lập trình viên. Guido van Rossum và nhóm phát triển Python đã tiếp tục cải tiến và phát triển ngôn ngữ, và phiên bản Python 1.0 chính thức được ra mắt vào tháng 1 năm 1994.

Trong thập kỷ tiếp theo, Python tiếp tục phát triển và trở nên ngày càng phổ biến. Phiên bản Python 2.0 được phát hành vào năm 2000, đánh dấu một bước tiến quan trọng trong việc cải thiện ngôn ngữ và thư viện chuẩn của nó. Python 2.0 đã đưa ra nhiều tính năng mới và cải tiến, bao gồm hỗ trợ Unicode và garbage collection tự động.

Tuy nhiên, đến năm 2008, Guido van Rossum và cộng đồng Python quyết định phát triển phiên bản Python 3.0. Phiên bản này có các thay đổi đáng kể so với Python 2.x, nhằm cải thiện sự nhất quán và sửa các vấn đề thiết kế của phiên bản trước. Thay đổi lớn nhất là việc thay đổi cú pháp và hạn chế sự không tương thích ngược với Python 2.x.

Python 3.0 được phát hành vào tháng 12 năm 2008 và được đánh giá là một bước tiến quan trọng trong sự phát triển của ngôn ngữ. Dần dần, cộng đồng Python

đã chuyển từ Python 2.x sang Python 3.x, và Python 2.x đã chính thức ngừng được hỗ trợ vào tháng 1 năm 2020.

Ngày nay, Python là một trong những ngôn ngữ lập trình phổ biến nhất trên thế giới. Nó được sử dụng rộng rãi trong nhiều lĩnh vực, bao gồm phân tích dữ liệu, trí tuệ nhân tạo, phát triển web, tự động hóa, và nhiều lĩnh vực khác. Cộng đồng Python vẫn tiếp tục phát triển và cải thiện ngôn ngữ, đồng thời phát triển các thư viện và framework phong phú để hỗ trợ các ứng dụng và dự án khác nhau.

1.3 Các tính năng của Python

- **Phân tích dữ liệu và học máy với mã Python**

Với lượng dữ liệu ngày càng tăng mà chúng ta có ngày nay, điều quan trọng là phải sử dụng nó trong quá trình ra quyết định. Với công nghệ máy học và phân tích dữ liệu, chúng ta có thể đạt được mục đích này. Bạn sẽ thích thú khi biết rằng ngôn ngữ lập trình Python là động lực đằng sau việc thu thập, tổ chức và thao tác dữ liệu này.

- **Phát triển web và phát triển phần mềm**

Ngôn ngữ Python cũng giúp phát triển web và phát triển phần mềm. Nó hỗ trợ nhiều khung như Pyramid, Django và Flask. Các khung này đã góp phần tạo ra các trang web nổi tiếng như Spotify, Reddit và Mozilla. Với quản lý nội dung, ủy quyền và truy cập cơ sở dữ liệu, các phiên bản khác nhau của ngôn ngữ lập trình Python là ngôn ngữ hàng đầu để phát triển web quyền và truy cập cơ sở dữ liệu, các phiên bản khác nhau của ngôn ngữ lập trình Python là ngôn ngữ hàng đầu để phát triển web.

- **Tự động hóa hoặc viết kịch bản bằng mã Python**

Tạo mã để hoàn thành các tác vụ tự động được gọi là viết kịch bản. Bạn có thể sử dụng Python cho mục đích này. Việc sử dụng kịch bản thường được thực hiện trong các công việc lặp đi lặp lại. Mã Python có thể giúp bạn tự động hóa quy trình. Nó giúp tiết kiệm thời gian, tài nguyên và công sức thủ công

- **Tự động hóa hoặc viết kịch bản bằng mã**

Python Tạo mã để hoàn thành các tác vụ tự động được gọi là viết kịch bản. Bạn có thể sử dụng Python cho mục đích này. Việc sử dụng kịch bản thường được thực hiện trong các công việc lặp đi lặp lại. Mã Python có thể giúp bạn tự động hóa quy trình. Nó giúp tiết kiệm thời gian, tài nguyên và công sức thủ công. Kiểm thử phần mềm và tạo nguyên mẫu Theo dõi lỗi, kiểm soát tòa nhà, phát triển web, phát triển phần mềm và kiểm tra phần mềm là những khía cạnh quan trọng mà Python có thể xử lý. Với mã Python để giải cứu bạn, thời gian cần thiết cho tất cả các hoạt động này có thể được giảm xuống mức tối thiểu, do đó cho phép bạn tập trung vào những gì cần thiết

1.4 Ứng dụng Pycharm

Pycharm là một nền tảng kết hợp được JetBrains phát triển như một IDE (Môi trường phát triển tích hợp) để phát triển các ứng dụng cho lập trình trong Python. Một số ứng dụng lớn như Tweeter, Facebook, Amazon và Pinterest sử dụng Pycharm để làm IDE Python của họ. Bài viết dưới đây sẽ giới thiệu chi tiết cho bạn về Pycharm cũng như hướng dẫn cách cài đặt và sử dụng Pycharm

1.5 Các tính năng của Pycharm

- **Trình chỉnh sửa mã thông minh:**
 - Giúp các lập trình viên viết mã chất lượng cao
 - Bao gồm các lược đồ màu cho các từ khóa, lớp và hàm. Điều này giúp tăng khả năng đọc và hiểu mã
 - Xác định lỗi một cách dễ dàng
 - Cung cấp tính năng tự động hoàn thiện và hướng dẫn hoàn thiện mã
- **Điều hướng mã**
 - Giúp các nhà phát triển trong việc chỉnh sửa và nâng cao mã với ít nỗ lực và thời gian hơn

Đồ án cơ sở 4

- Với việc điều hướng mã, nhà phát triển có thể dễ dàng điều hướng một lớp, hàm hoặc tệp

- Có thể xác định vị trí của một phần tử, một ký hiệu hoặc một biến trong mã nguồn trong thời gian ngắn khi sử dụng Pycharm

- Bằng việc sử dụng chế độ thấu kính, nhà phát triển có thể kiểm tra và gỡ lỗi toàn bộ mã nguồn.

- **Tái cấu trúc**

- Sử dụng Pycharm có lợi thế là thực hiện các thay đổi hiệu quả và nhanh chóng đối với cả biến cục bộ và biến toàn cục

- Tái cấu trúc trong Pycharm cho phép các nhà phát triển cải thiện cấu trúc bên trong mà không thay đổi hiệu suất bên ngoài của mã

- Nó cũng cho phép phân chia các lớp với các chức năng mở rộng hơn

CHƯƠNG 2: TỔNG QUAN VỀ TRÒ CHƠI CỜ VUA

2.1 Giới thiệu trò chơi

Cờ vua (tên tiếng Anh: Chess), trước kia còn được gọi là Cờ quốc tế, là trò chơi quốc tế và là môn thể thao trí tuệ cho 2 người chơi. Ngày nay, cờ vua là một trong những trò chơi phổ biến nhất thế giới với hàng triệu người tại nhà riêng, trong câu lạc bộ, trực tuyến, từ xa và trong các giải đấu. Trò chơi này diễn ra trên một bảng hình vuông, gọi là bàn cờ, gồm 8 hàng (đánh số từ 1 đến 8) và 8 cột (đánh các chữ cái từ a đến h), tạo ra 64 ô hình vuông với các màu đậm và nhạt xen kẽ nhau, với mỗi người chơi sẽ có ô màu nhạt ở hàng cuối cùng bên tay phải của mình khi ngồi vào bàn chơi cờ. Mỗi người sẽ bắt đầu ván cờ với 16 quân cờ và sẽ lần lượt đi các quân của mình sau khi đối phương đã đi xong một nước (hoàn thành nước đi).

Các quân cờ của mỗi bên bao gồm 8 Tốt, 2 Mã, 2 Tượng, 2 Xe, 1 Hậu và 1 Vua. Người cầm quân trắng luôn là người đi đầu tiên; người kia cầm quân đen. Các quân Hậu và Xe được gọi là quân nặng, còn Tượng và Mã được gọi là quân nhẹ.

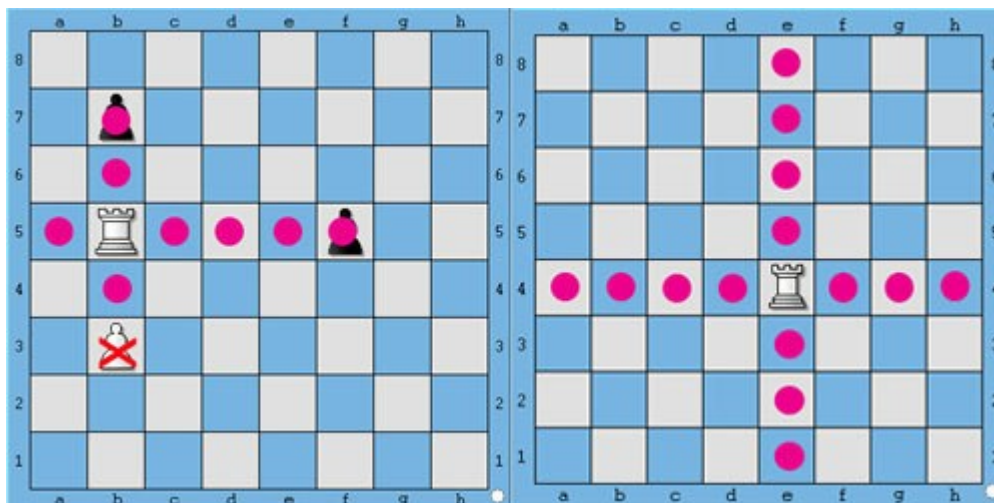
2.2 Các quy tắc của trò chơi

Khi một ván cờ vua bắt đầu, một người chơi sẽ cầm quân đen và người chơi còn lại cầm quân trắng. Việc chọn lựa ai sẽ cầm quân đen hay trắng phụ thuộc vào thể thức chơi là giải trí hay thi đấu trong hệ thống có tổ chức. Nó có thể là do hai người thỏa thuận (giải trí) hoặc do quyết định của trọng tài trong giải đấu. Bên cầm quân trắng luôn luôn được đi trước và do đó có ưu thế nhỏ so với bên cầm quân đen. Các quân cờ cần phải xếp trên bàn cờ tiêu chuẩn với ô nằm ở hàng cuối cùng bên tay phải người chơi bao giờ cũng có màu nhạt.

Đồ án cơ sở 4

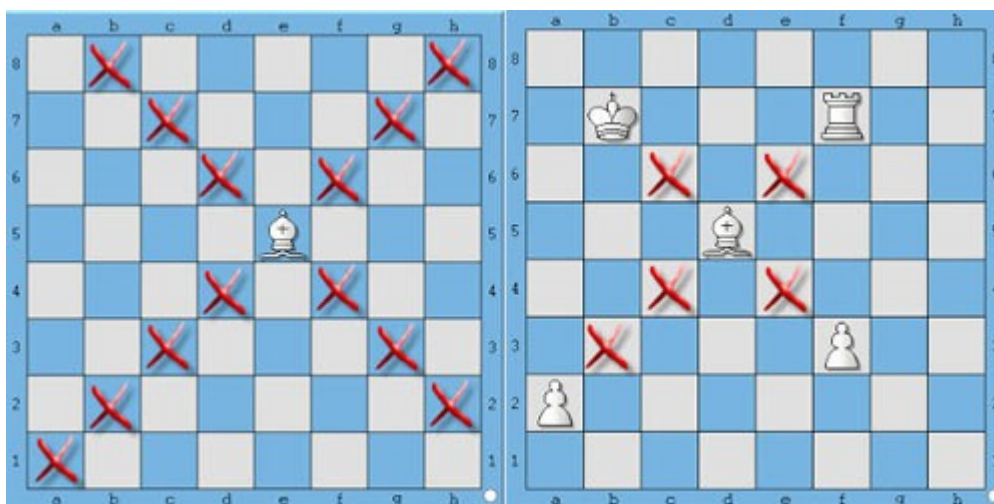
Các quân cờ có nước đi khác nhau:

Xe (ký hiệu quốc tế **R - Rook**) di chuyển theo các đường thẳng dọc theo cột hay hàng tới ô còn trống mà không có quân nào cản trên đường đi hay tới ô bị quân đối phương chiếm giữ (ăn quân) nhưng không thể vượt qua quân đang đứng ở ô đó. Ngoại lệ duy nhất là trường hợp nhập thành. Khi đó nó có thể nhảy qua quân Vua của mình để đứng cạnh nó. Chỉ có Xe mới có nước đi như vậy.



Hình 2: Minh họa nước đi quân Xe

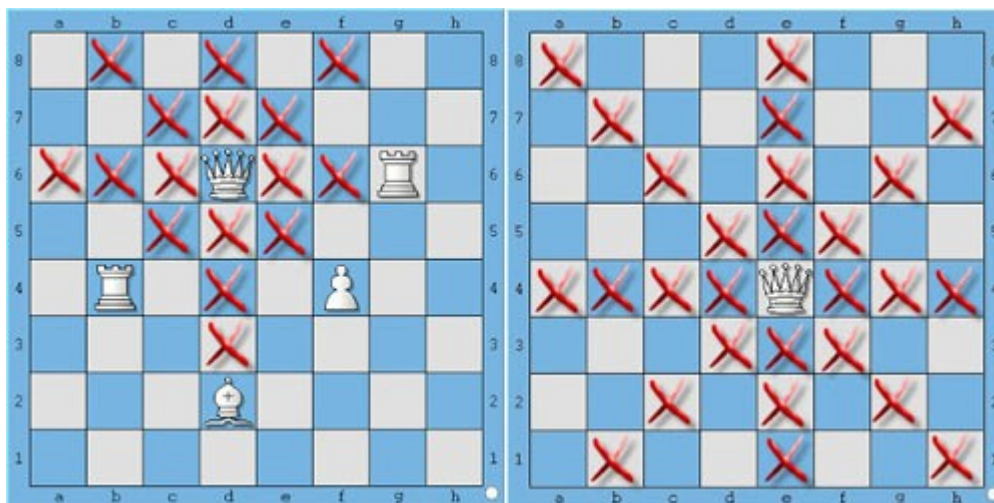
Tượng (ký hiệu quốc tế **B - Bishop**) di chuyển theo đường chéo tới ô có cùng màu với nguyên lý tương tự như Xe tới ô còn trống hay ô bị quân đối phương chiếm giữ (ăn quân).



Hình 3: Minh họa nước đi quân Tượng

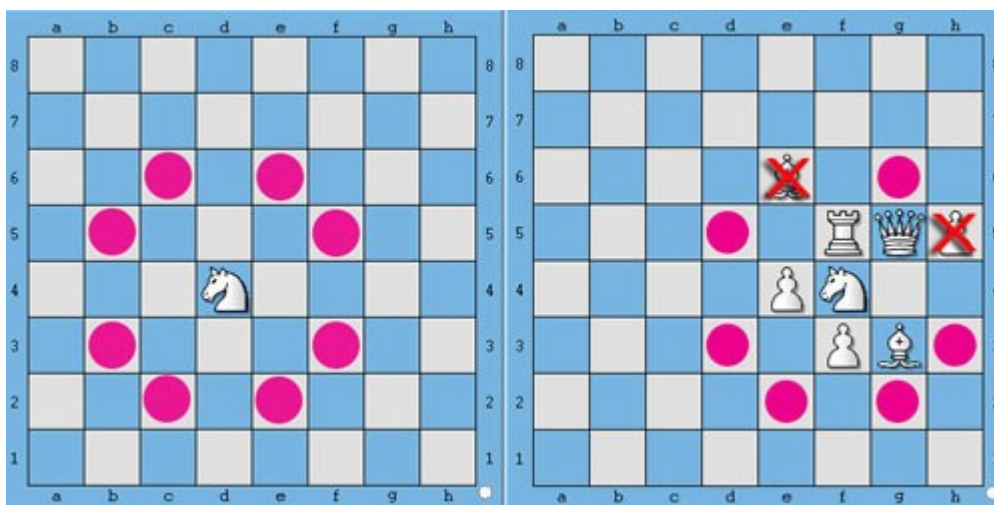
Đồ án cơ sở 4

Hậu (ký hiệu quốc tế **Q - Queen**) có nước đi là tổ hợp đơn giản của chuyển động của Xe và Tượng. Trong một nước đi nó có thể di chuyển theo đường chéo hoặc đường thẳng dọc theo cột hay hàng, với nguyên lý đi và ăn quân giống như Tượng và Xe.



Hình 4: Minh họa nước đi quân Hậu

Mã (ký hiệu quốc tế **N - Knight**) có thể di chuyển tới ô còn trống hay ô bị quân đối phương chiếm giữ (ăn quân) theo dạng hình chữ L (hình chữ nhật 3×2 hay 2×3). Quân Mã không bị cản như trong cờ tướng.

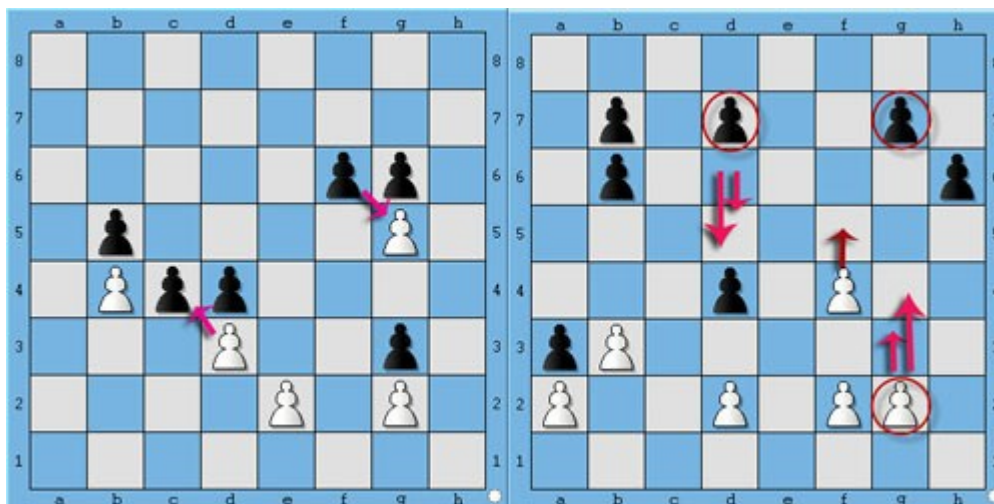


Hình 5: Minh họa nước đi quân Mã

Tốt (không cần ký hiệu) có thể di chuyển thẳng về phía trước chỉ một ô một lần tới ô còn trống (đi mà không ăn quân), nhưng khi di chuyển quân để ăn quân

Đồ án cơ sở 4

đối phương thì đi chéo. Ví dụ, Tốt trắng tại ô c4 có quyền ăn quân đối phương tại b5 hoặc d5 nếu một trong hai ô này có quân đối phương chiếm hoặc di chuyển xuống ô c5 nếu ô này còn trống, trừ hai trường hợp sau:

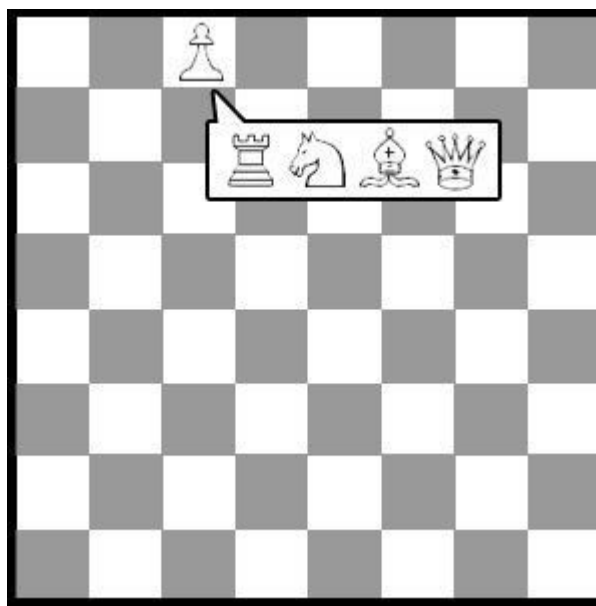


Hình 6: Minh họa nước đi quân Tốt

Nó có thể di chuyển 1 hoặc 2 ô nếu nó đi từ vị trí xuất phát ban đầu tới ô chưa bị chiếm giữ, nhưng không thể nhảy qua một quân khác để tới ô đó. Ví dụ Tốt trắng tại g2 có thể đi tới g3 hoặc g4 nếu đây là nước đi đầu tiên của nó và các ô này chưa bị chiếm giữ, nhưng nó không thể đi tới g4 nếu ô g3 đã có một quân nào đó chiếm giữ.

Trong trường hợp khi một quân Tốt nào đó của bên trắng đạt tới hàng 5 (ví dụ tới ô e5) và quân Tốt thuộc một trong hai cột của bên đen nằm ngay bên cạnh cột mà Tốt trắng này đang chiếm giữ (trong trường hợp đã cho là cột d và cột f) đi từ vị trí xuất phát đầu tiên (d7 hay f7) nhảy liên 2 ô tới ô d5/f5 thì Tốt trắng tại vị trí e5 ngay tại nước đi sau đó có quyền ăn Tốt đen tại ô d5/f5 và di chuyển tiếp tới ô d6/f6. Quyền này sẽ tự động mất, nếu tại nước đi ngay sau đó quân trắng di chuyển quân khác. Tương tự như vậy cho Tốt đen khi nó đã chiếm giữ hàng 4. Đây là trường hợp mà trong cờ vua người ta gọi là bắt tốt qua đường (en passant).

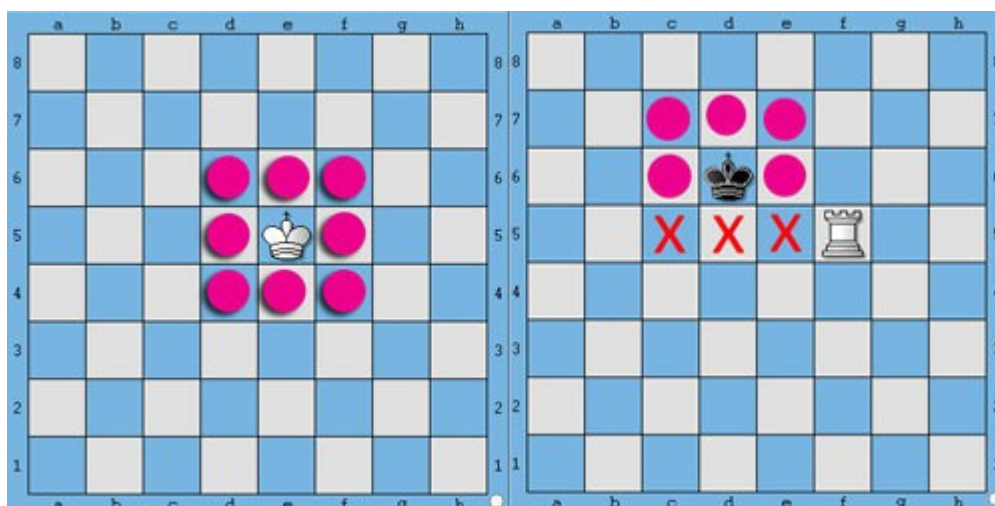
Tốt còn một đặc điểm nữa là khi nó di chuyển đến hàng cuối cùng thì người chơi có quyền phong cấp cho nó thành bất kỳ quân nặng hay nhẹ nào (Hậu, Xe, Tượng, Mã).



Hình 7: Minh họa quân tốt thăng cấp

Vua (ký hiệu quốc tế là **K - King**) là quân quan trọng nhất, nếu mất Vua thì người chơi thua cuộc. Mỗi lần đi nó có thể ăn quân hoặc di chuyển sang các ô bao quanh ô mà nó hiện tại đang chiếm giữ, nhưng không thể tới ô mà quân của mình đang chiếm giữ hay các ô bị quân đối phương kiểm soát. Ngoại lệ duy nhất là trường hợp nhập thành. Khi đó nó có thể di chuyển qua hai ô đồng thời với việc di chuyển quân Xe của mình để quân Xe đó đứng bên cạnh nó về phía cột trung tâm. Ký hiệu của nhập thành là 0-0 (nhập thành gần) và 0-0-0 (nhập thành xa). Xem thêm nhập thành.

Khi ăn quân đối phương, quân tấn công sẽ di chuyển tới ô đó và thay thế cho quân đối phương tại vị trí này, bắt tốt qua đường (en passant) là ngoại lệ duy nhất. Quân bị ăn được loại ra khỏi bàn cờ. Vua không thể không bảo vệ khỏi nước chiếu, do đó khi bị chiếu thì người chơi phải thực hiện các biện pháp nhằm cứu Vua (di chuyển Vua khỏi vị trí bị chiếu, ăn quân đang chiếu hay dùng quân khác của mình cản đường chiếu nếu có thể). Nếu không thể có nước đi để cứu Vua thì người chơi bị chiếu bí và thua cuộc.



Hình 8: Minh họa nước đi quân Vua

2.3 GDD (Game Design Document)

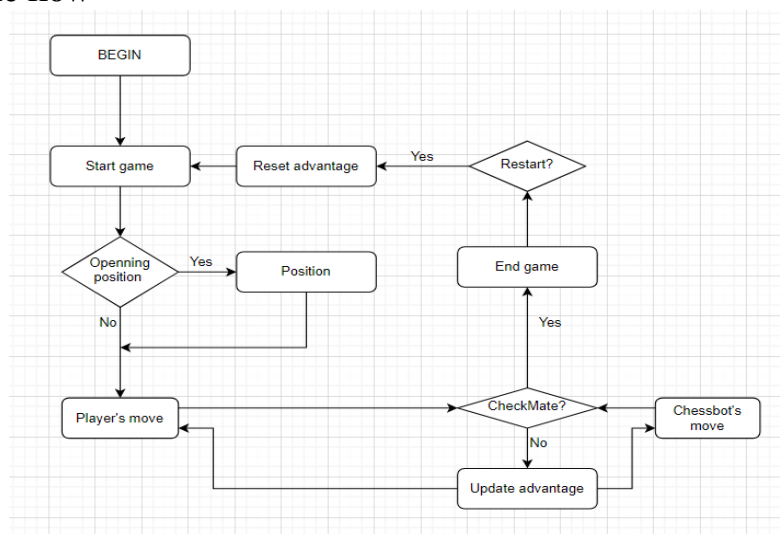
- **Gameplay :**

Bàn cờ vua 8x8 truyền thống và các quân cờ. Người di chuyển các quân cờ theo luật cờ vua, suy nghĩ sử dụng các chiến lược và chiến thuật để dành chiến thắng trước đối thủ (Chessbot).

- **Core loop:**

Từng bước dành chiến thắng bằng cách di chuyển các nước có lợi thế cho bản thân hoặc bất lợi cho đối thủ xuyên suốt cả ván đấu. Mỗi một nước đi có lợi, game sẽ thông báo số điểm lợi thế hiện tại trên giao diện, giúp người chơi dễ dàng theo dõi trạng thái hiện tại và suy nghĩ thế trận tiếp theo.

- **Game flow**



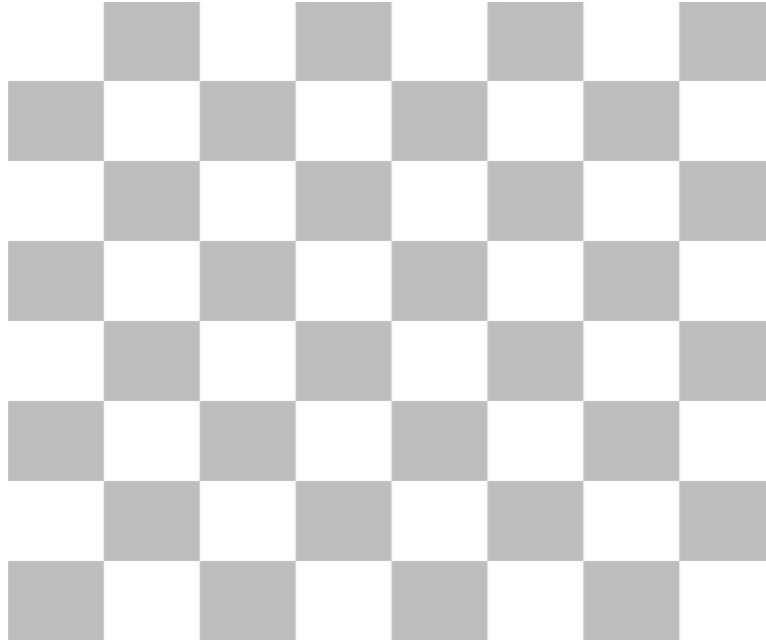
Hình 9: Game flow

Đồ án cơ sở 4

- **Object**

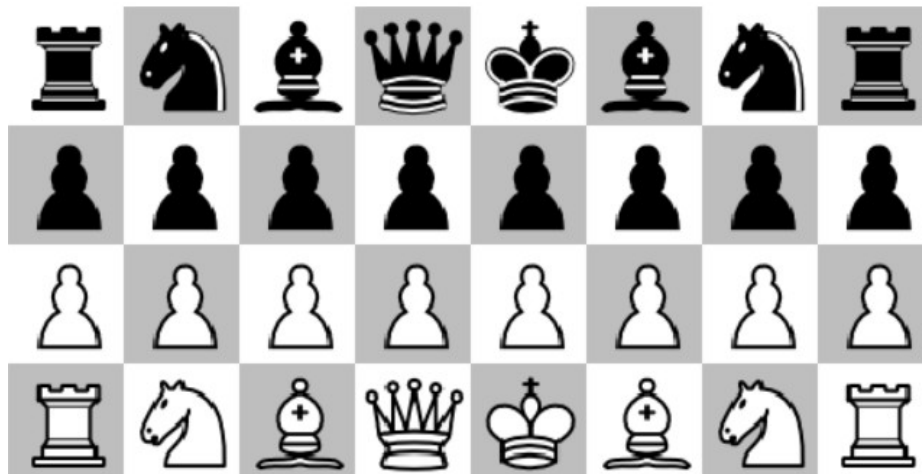
Các thành phần trong game được chia làm 2 phần chính: Bàn cờ và các thanh cài đặt.

Bàn cờ: Bàn cờ tiêu chuẩn 8x8



Hình 10: Bàn cờ tiêu chuẩn

Các quân cờ vua (đen và trắng)



Hình 11: Các quân cờ vua

CHƯƠNG 3. PHÂN TÍCH VÀ THIẾT KẾ GAME

3.1 Sơ đồ lớp

Game được xây dựng dựa trên các hàm chức năng là chính, vậy nên các đối tượng trong game sẽ rất tổng quát (ví dụ như bàn cờ là một đối tượng gồm các phương thức để tương tác với giao diện là chính, trong khi đó “điều khiển bàn cờ” lại là một đối tượng khác bao gồm các phương thức để điều khiển các quân cờ theo luật, cũng như kiểm tra trạng thái ván cờ hiện tại (chiếu, chiếu hết,...))

3.2 Cơ sở lý thuyết xây dựng Chess bot

3.2.1 Ý tưởng đưa ra nước đi tối ưu

Trong lý thuyết trò chơi, trò chơi có tổng bằng 0 là một biểu diễn toán học về tình huống trong đó mỗi người tham gia được hoặc mất tiện ích được cân bằng chính xác bởi những mất mát hoặc lợi ích của những người tham gia khác. Nếu tổng lợi ích của những người tham gia được cộng lại và tính tổng thiệt hại, cả hai sẽ có tổng bằng không. Cờ vua là một trò chơi có tổng bằng 0, nghĩa là cả hai bên đều có thể thắng, thua và hòa. Nếu người chơi A có thể thắng trong 1 nước đi thì nước đi đó chính là nước đi tốt nhất cho A. Nếu người B biết rằng có một nước đi mà dẫn đến tình huống người A có thể thắng ngay ở nước đi tiếp theo, trong khi nước đi khác thì sẽ dẫn đến tình huống mà người chơi A chỉ có thể tốt nhất là hòa, thì nước đi tốt nhất của người B chính là nước đi sau.

Theo lý thuyết này, bất kỳ lợi thế nào mà Người chơi A đạt được đều có nghĩa là bất lợi cho Người chơi B. Lợi thế có thể đến từ việc bắt quân của đối thủ hoặc có quân ở vị trí thuận lợi. Từ góc độ của Chess bot, khi bị mất quân đồng nghĩa với việc bị trừ điểm lợi thế, ăn được quân đồng nghĩa với cộng thêm điểm lợi thế. Từ ý tưởng này, ta sẽ xây dựng các thuật toán đánh giá lợi thế dựa trên mỗi nước đi cho Chess bot, khi tính toán đến việc đi một nước đi nào đó, Chess bot sẽ xem xét nước đi đó có lợi hay bất lợi trong tương lai (tùy theo độ sâu), từ đó đưa ra nước đi thích hợp.

3.2.2 Các thuật toán để đưa ra nước đi tối ưu

Dựa vào ý tưởng trên, các thuật toán cần có để đưa ra được nước đi tối ưu cho Chess bot bao gồm:

- Lượng giá quân cờ, đánh giá thế trận hiện tại dựa trên vị trí các quân cờ: Đơn vị đo chính cho các thuật toán khác vì mục đích cuối cùng của Chess bot là đưa ra được nước đi có lợi thế nhiều nhất.

- Thuật toán tìm kiếm đệ quy mở rộng không gian trạng thái: Với mỗi một nước đi của bản thân hoặc đối thủ, cần tính toán đến những khả năng tiếp theo trong tương lai mà đối thủ hoặc bản thân có thể đi tiếp (trạng thái) để đánh giá các trạng thái xem trạng thái nào ít tổn thất nhất. Chess bot cần phải tính toán đến các nước đi có thể của đối thủ cùng các khả năng có thể xảy ra trong tương lai thông qua không gian trạng thái, giúp cho nó có thể đưa ra nước đi tốt do đã tính toán được trước các lợi thế có thể có một cách tương đối sâu.

- Thuật toán Negamax: Dựa trên những ý tưởng trên, cần có một thuật toán để có thể “tối đa hóa lợi thế” và “tối thiểu hóa bất lợi”. Giải thuật Negamax giúp tìm ra nước đi tốt nhất, bằng cách đi ngược từ cuối trò chơi trở về đầu. Tại mỗi bước, nó sẽ ước định rằng người A đang cố gắng tối đa hóa cơ hội thắng của A khi đến phiên anh ta, còn ở nước đi kế tiếp thì người chơi B cố gắng để tối thiểu hóa cơ hội thắng của người A (nghĩa là tối đa hóa cơ hội thắng của B). Hơn nữa, người A và người B phải tính toán được đến cả những nước đi có thể khiến bản thân rơi vào tình huống bất lợi nhất, hay gọi là tối thiểu hóa cơ hội thắng của chính mình, vì đó là nước đi mà đối thủ muốn đi nhất và phải tránh được điều đó xảy ra. Thuật toán Negamax sẽ đưa ra trạng thái hoặc là tốt nhất cho bản thân và tồi tệ nhất cho đối thủ.

- Thuật toán cắt tỉa Alpha-Beta: Như đã nói, Chessbot cần phải sinh ra các không gian trạng thái bằng giải thuật đệ quy và đánh giá chúng bằng Negamax. Hiện tại, nó đã có thể đưa ra những quyết định đúng đắn và hợp lý. Tuy nhiên, độ sâu tìm kiếm trạng thái càng cao, thời gian tìm kiếm sẽ càng tăng lên (chỉ cần tìm

kiểm với độ sâu là 4 thì thời gian có thể lên đến hàng trăm giây). Cắt tỉa alpha-beta giúp cải thiện hiệu quả của thuật toán bằng cách 'cắt tỉa' các nhánh mà Chessbot không cần đánh giá.

3.3 Thiết kế logic cho Chess bot

3.3.1 Hệ số lực của quân cờ

Để Chessbot đưa ra một nước đi đủ tốt, nó cần một “con số” để hiểu được trạng thái bàn cờ hiện tại, chính xác hơn là một chức năng đánh giá. Về cơ bản, ta muốn gán một 'điểm số' cho từng trường hợp bàn cờ (tức là từng tập hợp vị trí của các quân cờ trên bàn cờ) để Chessbot có thể đưa ra quyết định về vị trí nào thuận lợi hơn các vị trí khác.

Khía cạnh đầu tiên và là nguyên tử cho quá trình đánh giá liên quan đến việc ấn định trọng số cho từng quân cờ. Nếu Chessbot chơi từ góc nhìn của quân đen, vậy thì bất kỳ quân đen nào sẽ cộng vào điểm số lợi thế của Chessbot, trong khi bất kỳ quân trắng nào sẽ trừ vào điểm lợi thế, theo các trọng số sau, tạm gọi là hệ số lực của mỗi quân cờ:

Tốt (Pawn): 1

Mã (Knight): 3

Tượng (Bishop): 3

Xe (Rook): 5

Hậu (Queen): 9

Vua (King): 0

```
piece_score = {"K": 0, "Q": 9, "R": 5, "B": 3, "N": 3, "p": 1}
```

Hình 12: Hệ số lực của mỗi quân cờ

Các quân cờ đã được “đánh giá sức mạnh”, tuy nhiên, một quân cờ khi ở một trí cụ thể trên bàn cờ sẽ mạnh hơn ở các vị trí khác, ví dụ quân Mã trắng khi ở

Đồ án cơ sở 4

f3 có thể kiểm soát được 6 ô, và quan trọng nhất là các ô trung tâm, từ đó làm chậm các cuộc tấn công của quân đen vào khu trung tâm. Rõ ràng quân Mã trắng khi đứng ở f3 mạnh hơn hẳn so với việc nó đứng ở g1 (chỉ kiểm soát được 2 ô ở góc bàn cờ). Vậy nên, cần đánh giá một quân cờ không chỉ ở giá trị lực của bản thân nó, mà phải đánh giá cả việc nó đang đứng ở đâu trên bàn cờ, vị trí đó tăng bao nhiêu lợi thế cho người chơi và Chessbot. Các giá trị sau đây thể hiện giá trị cộng thêm của một quân cờ khi đứng ở một vị trí cụ thể (đối với quân trắng), được gọi là bảng giá trị vị trí (Piece Position Score), các giá trị này và giá trị các quân cờ đã trình bày ở trên được tham khảo dựa theo phần mềm chơi cờ vua nổi tiếng Sunfish Engine:

```
knight_scores = [[0.0, 0.1, 0.2, 0.2, 0.2, 0.2, 0.1, 0.0],
                  [0.1, 0.3, 0.5, 0.5, 0.5, 0.5, 0.3, 0.1],
                  [0.2, 0.5, 0.6, 0.65, 0.65, 0.6, 0.5, 0.2],
                  [0.2, 0.55, 0.65, 0.7, 0.7, 0.65, 0.55, 0.2],
                  [0.2, 0.5, 0.65, 0.7, 0.7, 0.65, 0.5, 0.2],
                  [0.2, 0.55, 0.6, 0.65, 0.65, 0.6, 0.55, 0.2],
                  [0.1, 0.3, 0.5, 0.55, 0.55, 0.5, 0.3, 0.1],
                  [0.0, 0.1, 0.2, 0.2, 0.2, 0.2, 0.1, 0.0]]

bishop_scores = [[0.0, 0.2, 0.2, 0.2, 0.2, 0.2, 0.2, 0.0],
                  [0.2, 0.4, 0.4, 0.4, 0.4, 0.4, 0.4, 0.2],
                  [0.2, 0.4, 0.5, 0.6, 0.6, 0.5, 0.4, 0.2],
                  [0.2, 0.5, 0.5, 0.6, 0.6, 0.5, 0.5, 0.2],
                  [0.2, 0.4, 0.6, 0.6, 0.6, 0.6, 0.4, 0.2],
                  [0.2, 0.6, 0.6, 0.6, 0.6, 0.6, 0.6, 0.2],
                  [0.2, 0.5, 0.4, 0.4, 0.4, 0.4, 0.5, 0.2],
                  [0.0, 0.2, 0.2, 0.2, 0.2, 0.2, 0.2, 0.0]]

rook_scores = [[0.25, 0.25, 0.25, 0.25, 0.25, 0.25, 0.25, 0.25],
                [0.5, 0.75, 0.75, 0.75, 0.75, 0.75, 0.75, 0.5],
                [0.0, 0.25, 0.25, 0.25, 0.25, 0.25, 0.25, 0.0],
                [0.0, 0.25, 0.25, 0.25, 0.25, 0.25, 0.25, 0.0],
                [0.0, 0.25, 0.25, 0.25, 0.25, 0.25, 0.25, 0.0],
                [0.0, 0.25, 0.25, 0.25, 0.25, 0.25, 0.25, 0.0],
                [0.0, 0.25, 0.25, 0.25, 0.25, 0.25, 0.25, 0.0],
                [0.25, 0.25, 0.25, 0.5, 0.5, 0.25, 0.25, 0.25]]

queen_scores = [[0.0, 0.2, 0.2, 0.3, 0.3, 0.2, 0.2, 0.0],
                 [0.2, 0.4, 0.4, 0.4, 0.4, 0.4, 0.4, 0.2],
                 [0.2, 0.4, 0.5, 0.5, 0.5, 0.5, 0.4, 0.2],
                 [0.3, 0.4, 0.5, 0.5, 0.5, 0.5, 0.5, 0.3],
                 [0.4, 0.4, 0.5, 0.5, 0.5, 0.5, 0.4, 0.3],
                 [0.2, 0.5, 0.5, 0.5, 0.5, 0.5, 0.4, 0.2],
                 [0.2, 0.4, 0.5, 0.4, 0.4, 0.4, 0.4, 0.2],
                 [0.0, 0.2, 0.2, 0.3, 0.3, 0.2, 0.2, 0.0]]

pawn_scores = [[0.8, 0.8, 0.8, 0.8, 0.8, 0.8, 0.8, 0.8],
                [0.7, 0.7, 0.7, 0.7, 0.7, 0.7, 0.7, 0.7],
                [0.3, 0.3, 0.4, 0.5, 0.5, 0.4, 0.3, 0.3],
                [0.25, 0.25, 0.3, 0.45, 0.45, 0.3, 0.25, 0.25],
                [0.2, 0.2, 0.2, 0.4, 0.4, 0.2, 0.2, 0.2],
                [0.25, 0.15, 0.1, 0.2, 0.2, 0.1, 0.15, 0.25],
                [0.25, 0.3, 0.3, 0.0, 0.0, 0.3, 0.3, 0.25],
                [0.2, 0.2, 0.2, 0.2, 0.2, 0.2, 0.2, 0.2]]
```

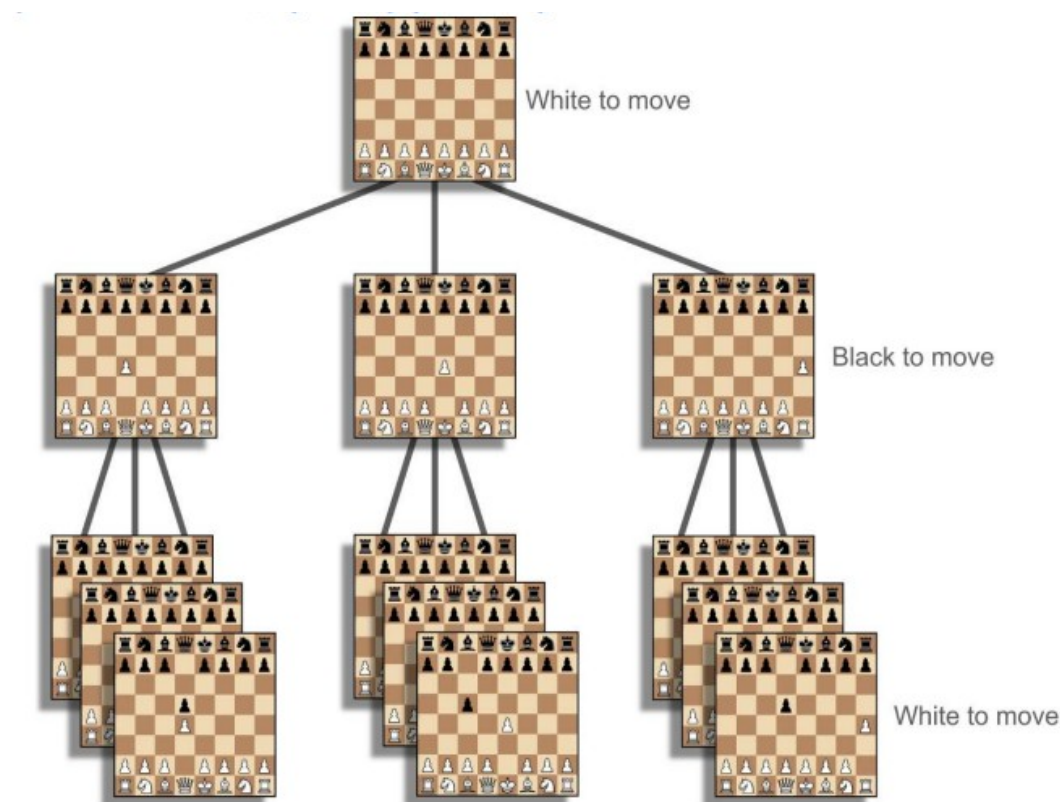
Hình 13: Bảng giá trị vị trí

- Đối với quân đen thì chỉ cần đảo ngược các giá trị lại:

```
piece_position_scores = {"wN": knight_scores,
                        "bN": knight_scores[::-1],
                        "wB": bishop_scores,
                        "bB": bishop_scores[::-1],
                        "wQ": queen_scores,
                        "bQ": queen_scores[::-1],
                        "wR": rook_scores,
                        "bR": rook_scores[::-1],
                        "wp": pawn_scores,
                        "bp": pawn_scores[::-1]}
```

Hình 14: Giá trị vị trí của quân cờ

3.3.2 Cây trò chơi



Hình 15: Minh họa cây trò chơi

Cây trò chơi (game tree) là một cách biểu diễn trạng thái và các lựa chọn có thể có trong một trò chơi. Trong trường hợp của cờ vua, cây trò chơi biểu diễn tất cả các nước đi có thể được thực hiện từ trạng thái hiện tại của bàn cờ. Mỗi nút trong cây trò chơi đại diện cho một trạng thái của bàn cờ, và các cạnh đại diện cho

các nước đi có thể được thực hiện từ trạng thái đó. Từ trạng thái hiện tại, có thể có rất nhiều lựa chọn khác nhau về các nước đi, do đó cây trò chơi sẽ có rất nhiều nhánh. Một cây trò chơi cho cờ vua sẽ bao gồm tất cả các trạng thái có thể có trên bàn cờ và các nước đi có thể được thực hiện từ mỗi trạng thái đó. Các nút lá của cây trò chơi sẽ đại diện cho các trạng thái mà một bên đã chiến thắng hoặc là hòa. Mục tiêu của việc tìm kiếm cây trò chơi là tìm ra nước đi tốt nhất từ trạng thái hiện tại đến một trạng thái chiến thắng hoặc hòa. Đó chính là mục đích chính của việc lựa chọn ra một nước đi tối ưu.

3.3.3 Thuật toán Negamax

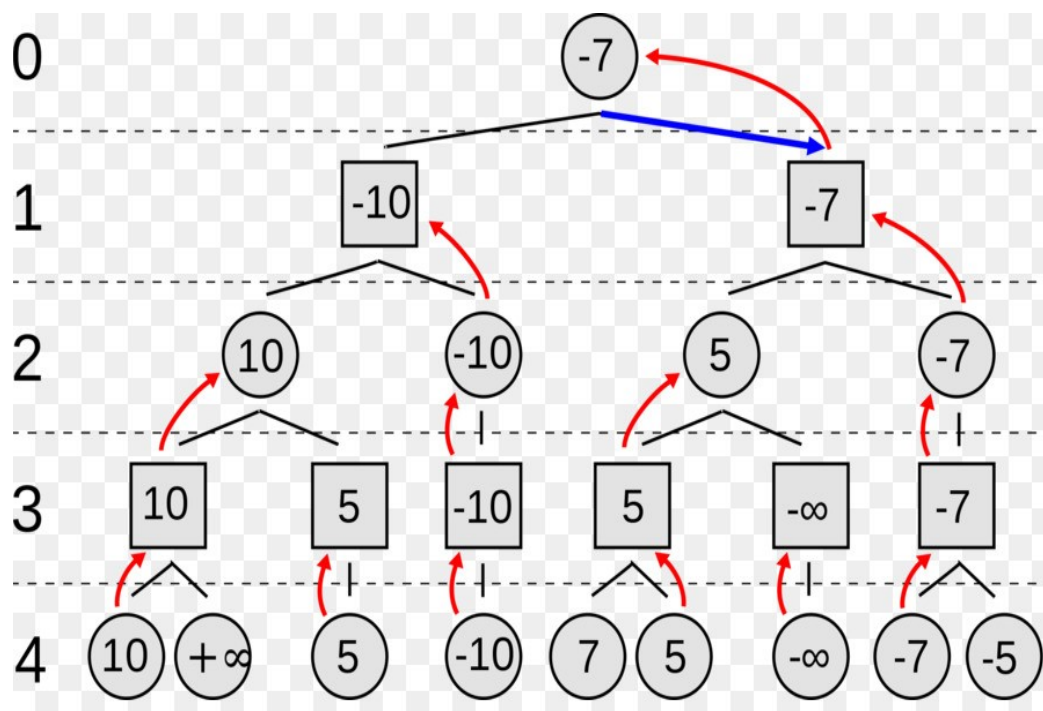
Trong quá trình chơi, cả hai đấu thủ đều cố gắng đi như thế nào đó để đạt điểm tuyệt đối lớn nhất. Người chơi cực đại sẽ tìm những nước đi dẫn đến điểm của mình cao nhất có thể (hay chính là đầy điểm của người chơi cực tiểu bớt âm hơn). Trái lại, người chơi cực tiểu cố đi để đạt tới vị trí mà điểm của mình nhỏ nhất có thể.

Từ đó ta hình thành tư tưởng thuật toán Negamax như sau: bắt đầu ở một nút (một thế cờ), ta thực hiện tất cả các nước có thể đi, thăm toàn bộ các nút con của nó trong cây trò chơi. Sau khi đã thăm toàn bộ các nút, kiểm tra xem hiện tại là lượt chơi của đấu thủ nào.

Nếu đang là lượt chơi của đấu thủ maximizer, ta xét toàn bộ các nút con của nó, chọn ra nút con có giá trị lớn nhất.

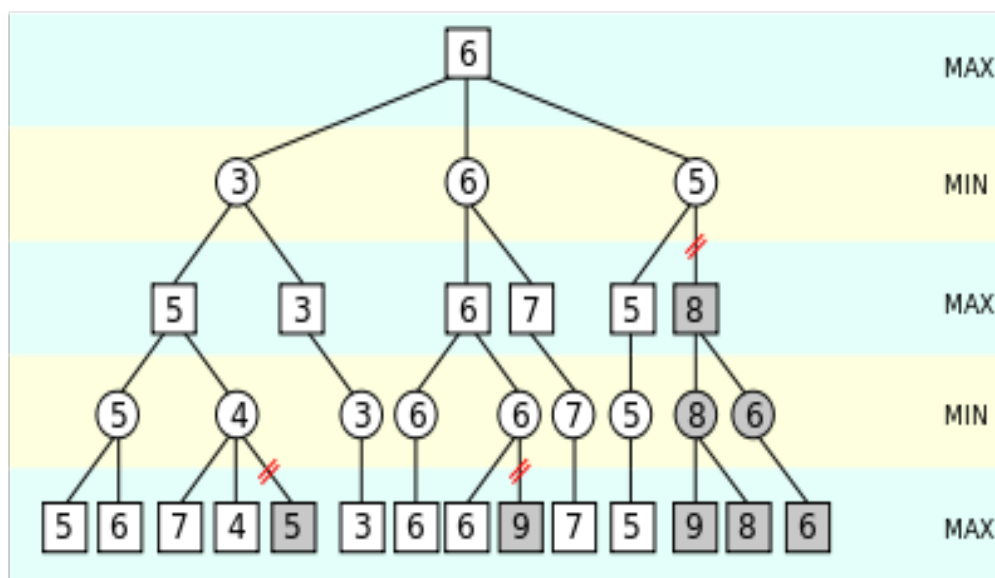
Nếu đang là lượt chơi của đấu thủ minimizer, ta xét toàn bộ các nút con của nó, chọn ra nút con có giá trị nhỏ nhất.

Toàn bộ ý tưởng của thuật toán này là dựa trên việc chuyển đổi mỗi thế cờ thành một con số để đánh giá. Rất tiếc là các con số này thường không tốt và không đủ để đánh giá hết mọi điều. Mặt khác, thuật toán này có thể rất tốn kém (chạy chậm) do việc sinh các nước đi và lượng giá rất tốn thời gian tính toán, do vậy độ sâu của cây trò chơi cũng bị hạn chế nhiều. Ta cần có thêm những cải tiến để cải thiện tình hình.



Hình 16: Minh họa thuật toán Negamax

3.3.4 Cắt tỉa Alpha-Beta



Hình 17: Minh họa về thuật toán cắt tỉa Alpha-Beta

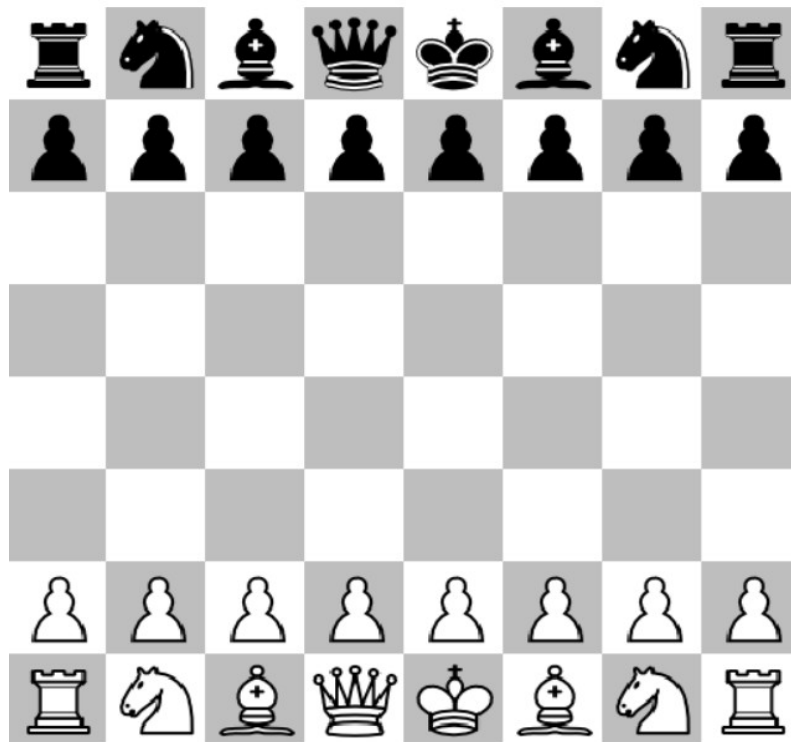
Như đã nói, chỉ cần thuật toán minimax hoạt động đúng, Chessbot hiện tại đã có thể đưa ra những nước đi khá chính xác. Tuy nhiên, do là thuật toán đệ quy, cùng với việc sinh cây trò chơi dựa trên mọi nước đi có thể, vậy nên khi độ sâu càng lớn thì thời gian tìm kiếm sẽ càng tăng lên. Người chơi không thể chờ đợi

hàng phút để Chessbot đưa ra quyết định. Để rút ngắn thời gian tính toán, ta cần bỏ đi các trường hợp mà “dù có đi tiếp cũng sẽ không bao giờ tìm được nước đi nào tốt hơn”. Cắt tỉa alpha-beta sẽ lược bỏ các nhánh mà Chessbot không cần đánh giá đến, nhằm nâng cao hiệu suất, giảm dung lượng tài nguyên và cải thiện tốc độ tính toán của Chessbot.

Ý tưởng chính của thuật toán Alpha-Beta: Thuật toán duy trì hai giá trị, alpha và beta, tương ứng đại diện cho số điểm tối thiểu mà “người chơi tối đa hóa” đang có và số điểm tối đa mà “người chơi tối thiểu hóa” đang có, hay ngắn gọn alpha là cơ hội chiến thắng của người A và beta là nguy cơ thua cuộc của người B. Ban đầu, alpha là âm vô cùng và beta là dương vô cùng, tức là cả hai người chơi đều bắt đầu với số điểm kém nhất có thể của họ. Bất cứ khi nào số điểm tối đa mà người chơi tối thiểu hóa (tức là người chơi "beta") đang có thấp hơn số điểm tối thiểu mà người chơi tối đa hóa (tức là người chơi "alpha") đang có (tức là $\beta < \alpha$), thì thuật toán sẽ bỏ qua, không cần đánh giá trạng thái này vì nếu từ trạng thái mà sinh ra tiếp các trạng thái tiếp theo thì nước đi sẽ luôn luôn tệ hơn.

CHƯƠNG 4. THIẾT KẾ GIAO DIỆN

4.1 Giao diện trò chơi



Hình 18: Giao diện trò chơi

4.2 Giao diện thông báo không có quân cờ được chọn

Trò chơi sẽ hiển thị thông báo không hợp lệ nếu người chơi chọn sai nước đi cho quân cờ được chọn



Hình 19: Giao diện thông báo không có quân cờ được chọn

4.3 Giao diện thông báo nước đi không hợp lệ



Hình 19: Giao diện thông báo nước đi không hợp lệ

4.4 Giao diện chiến thắng

Người chơi chiếu tướng và giành chiến thắng.



Hình 20: Giao diện chiến thắng

4.5 Giao diện gợi ý nước đi

Khi người dùng nhấp vào quân cờ thì bàn cờ sẽ hiện lên màu sắc ở ô mà quân cờ đang đứng và những ô mà quân cờ có thể di chuyển đến



Hình 21: Giao diện gợi ý nước đi

CHƯƠNG 5: KẾT LUẬN

5.1 Ưu điểm

- Nhìn chung, Chessbot có thể đưa ra các nước đi khá chính xác và nhanh chóng nhờ 2 thuật toán Negamax và cắt tỉa Alpha – beta.
- Kết quả lý thuyết đúng tuyệt đối: Độ sâu cây tìm kiếm là hữu hạn, vậy nên sử dụng thuật toán Negamax cho Chessbot là một giải pháp tối ưu về mặt lý thuyết khi nó có thể đánh giá tất cả các nốt trên cây trò chơi hiện tại để tìm ra nước đi tốt nhất, ko cần xét đến thứ tự các nốt.
- Hiệu suất tốt: Cắt tỉa Alpha – beta cho phép giảm bớt các trường hợp không cần tiếp tục tính toán trên cây trò chơi, giúp Chessbot tối ưu về mặt hiệu suất tính toán, giúp nó đưa ra kết quả nhanh hơn và tiết kiệm tài nguyên hơn.

5.2 Nhược điểm

Mặc dù Chessbot đã có thể hoạt động tốt, tuy nhiên nó vẫn còn một vài nhược điểm nhất định cho các tính chất đặc trưng của 2 thuật toán chính:

- Thời gian tính toán: Thuật toán Negamax và Alpha-beta có thể mất nhiều thời gian để tìm kiếm toàn bộ cây trò chơi. Đặc biệt là khi phải đánh giá số lượng lớn các nước đi khả thi khi độ sâu cây tìm kiếm tăng lên.
- Hạn chế của hàm đánh giá: Thuật toán Negamax và Alpha-beta yêu cầu một hàm heuristic đủ chính xác để ước lượng giá trị của các quân cờ trong trạng thái ván cờ hiện tại. Tuy nhiên, việc thiết kế một hàm đánh giá chính xác và đủ mạnh để mô tả toàn bộ các tình huống trong ván cờ là một thách thức rất lớn.

5.3 Hướng phát triển

- Xây dựng trò chơi online thông qua mạng lan.
- Xây dựng cửa sổ chat giữa 2 người chơi

TÀI LIỆU THAM KHẢO

- [1] Giáo trình Artificial Intelligence – Stuart Russell, Peter Norvig 1994.
- [2]] “Trí tuệ nhân tạo” , Đinh Mạnh Tường, NXB KHKT, 2002.
- [3] “Trí tuệ nhân tạo” , Nguyễn Thanh Thủy, NXB Giáo dục, 1997.
- [4] Tên giáo trình: "Artificial Intelligence for Games" (Trí tuệ nhân tạo cho trò chơi) Tác giả: Ian Millington và John Funge Năm xuất bản: 2009 Chương: Chương 33 - "Board Games"